Test Case 1 Login and authentication

Test Case 1	
Project Name: PLAYSPORT	
Login and authentication Test Case	
Test Case ID: PS_TC_01	**Test Designed By:** Sidharth Suppoodan
Test Priority: High	**Test Designed Date:** 22/10/2026
Module: Login and authentication	**Test Executed By:** Automated test runner
Test Title: Login, MFA challenge and session isolation	**Test Execution Date:** 24/10/2026
Description: Exercise the login UI and account changes with controlled responses across roles (Customer, Admin), validate credential verification, JWT token issuance, and session error responses.	
Pre-Condition: FastAPI backend and test fixtures active; seeded valid customer and admin accounts; valid and rejected API responses.	

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Submit valid login once	Eligible fixture account: arjun.nair@example.com	One request; success callback with JWT Bearer token	Login callback assertion passed; role CUSTOMER verified	**PASS**
2	Return admin challenge	Admin credentials: admin@playsport.com	HTTP 200 OK; Return admin token; role verified as ADMIN	Admin callback assertion passed; ALEX ADMIN access granted	**PASS**
3	Reject credentials	Error response fixture: WrongPassword!	HTTP 401 Unauthorized; Display error; do not report success	Failure message assertion passed; no token issued	**PASS**
4	Submit empty login form in UI	Empty input fields	Halt submission; display inline validation error	Validation error assertion passed; submission halted	**PASS**
5	Switch account during session read	Two account identities (Customer and Owner)	Clear private cache; restrict route access strictly by RBAC	Account isolation assertion passed; state segmented	**PASS**

Post-Condition: Client authentication behavior verified with mocks and live services; JWT bearer session established.

Test Case 2 Turf Slot Reservation and Concurrency Control

Test Case 2	
Project Name: PLAYSPORT	
Turf Booking & Concurrency Collision Test Case	
Test Case ID: PS_TC_02	**Test Designed By:** Sidharth Suppoodan
Test Priority: High	**Test Designed Date:** 22/10/2026
Module: Booking Engine & Concurrency Guard	**Test Executed By:** Automated test runner
Test Title: Atomic Slot Reservation and Double-Booking Prevention	**Test Execution Date:** 24/10/2026
Description: Verify that database-level pessimistic row locking (SELECT ... FOR UPDATE) within an atomic transaction prevents two concurrent customers from booking the exact same pitch slot.	
Pre-Condition: Active turf ground with batch-generated available time slots; two distinct authenticated customer sessions (Customer 1 & Customer 2).	

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Batch-generate slots as facility owner	turf_id: 1, ground_id: 1 hours: 14:00 - 18:00	HTTP 200 OK; Consecutive hourly slot records created with status AVAILABLE	HTTP 200; Slots generated and available in pitch schedule	**PASS**
2	Customer 1 books available time slot	Customer 1 Token target_slot_id: 101	HTTP 201 Created; Status CONFIRMED; Booking ref starts with 'STC-'	HTTP 201 Created; Reference 'STC-XXXXXXXX' generated; Slot status BOOKED	**PASS**
3	Customer 2 attempts collision booking on same slot	Customer 2 Token identical slot_id: 101	HTTP 409 Conflict; Reject transaction; detail: Double-booking prevented	HTTP 409 Conflict; Atomic rollback executed; collision prevented	**PASS**
4	Query slot availability status	GET /api/slots/by-ground/1	Target slot marked as BOOKED; excluded from available inventory	Target slot confirmed unavailable; zero data corruption	**PASS**

Post-Condition: Only one valid booking entry is retained in database; composite unique constraint (ground_id, slot_date, start_time) remains unviolated.

Test Case 3 End-to-End Booking Workflow (Selenium WebDriver)

Test Case 3	
Project Name: PLAYSPORT	
End-to-End Booking Workflow Test Case	
Test Case ID: PS_TC_03	**Test Designed By:** Sidharth Suppoodan
Test Priority: High	**Test Designed Date:** 22/10/2026
Module: Customer Booking Workflow	**Test Executed By:** Automated test runner (Selenium E2E)
Test Title: Client-Side Slot Selection, Auth Intercept & Booking Celebration	**Test Execution Date:** 24/10/2026
Description: Validate the complete browser journey: browsing turf listings, viewing pitch schedules, selecting hourly slots, completing authentication challenge, and displaying the booking celebration modal.	
Pre-Condition: React frontend running on localhost:5173, FastAPI backend on localhost:8000; seeded PlayZone Arena turf with available slots.	

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Navigate to Turfs listing and select venue	URL: /turfs Target: PlayZone Arena	Browser routes to venue detail page; Pitch tabs and slot grid rendered	Venue detail loaded; Hourly slot badges visible in DOM	**PASS**
2	Select available time slot badge	Click ':00' slot button	Slot active styling applied; 'Proceed to Booking' action button activates	Button highlighted; Total checkout price reflected in summary	**PASS**
3	Trigger unauthenticated checkout check	Click 'Proceed to Booking'	Intercept checkout; Display Sign In authentication modal dialog	Modal dialog rendered on screen with credential inputs	**PASS**
4	Authenticate credentials in modal	arjun.nair@example.com Customer@123	Validate credentials; Dismiss modal; Restore active booking state	Modal closed; User authenticated without losing chosen slot	**PASS**
5	Confirm reservation and trigger celebration	Click final 'Proceed to Booking'	Booking submitted; 'Booking Confirmed!' modal displayed with confetti	Modal verified in DOM containing 'Booking Confirmed!'	**PASS**

Post-Condition: Confirmed booking record stored in database; reference displayed to customer; slot updated to booked in UI.

Test Case 4 Open Match Matchmaking and Capacity Clamping

Test Case 4					
Project Name: PLAYSPORT					
Open Match Matchmaking Test Case					
Test Case ID: PS_TC_04			**Test Designed By:** Sidharth Suppoodan		
Test Priority: Medium			**Test Designed Date:** 22/10/2026		
Module: Athlete Matchmaking & Open Matches			**Test Executed By:** Automated test runner		
Test Title: Open Match Creation, Player Joining & Dynamic Capacity Clamping			**Test Execution Date:** 24/10/2026		
Description: Validate that social matchmaking lobbies dynamically track joined participants, lock status transitions when maximum capacity is reached, and prevent player oversubscription.					
Pre-Condition: Active database session; match creator account and two distinct athlete profiles (Player A, Player B).					
#	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status**
1	Create Open Match lobby with capacity limit	sport: Badminton max_players: 2	HTTP 201 Created; Lobby initialized with status OPEN, current_players: 0	HTTP 201 Created; Match ID generated with max_players = 2	**PASS**
2	First athlete joins open lobby	rahul.kumar@example.com	HTTP 200 OK; current_players increments to 1; status remains OPEN	HTTP 200; current_players = 1; status = OPEN confirmed	**PASS**
3	Second athlete joins and fills lobby capacity	sarah.j@example.com	HTTP 200 OK; current_players increments to 2; status transitions to FULL	HTTP 200; current_players = 2; status automatically converted to FULL	**PASS**
4	Third athlete attempts to join full lobby	arjun.nair@example.com	HTTP 400 Bad Request; Join rejected; Detail indicates match full	HTTP 400 Bad Request returned; Roster clamped; Zero oversubscription	**PASS**
Post-Condition: Match lobby capacity is strictly clamped to max_players; roster list accurately stores player foreign keys.					

Test Case 5 Facility Turf Registration and Admin Approval

Test Case 5	
Project Name: PLAYSPORT	
Facility Onboarding & Admin Moderation Test Case	
Test Case ID: PS_TC_05	**Test Designed By:** Sidharth Suppoodan
Test Priority: Medium	**Test Designed Date:** 22/10/2026
Module: Turf Owner & Super Admin Moderation	**Test Executed By:** Automated test runner
Test Title: Turf Registration Submission, Pending Queue Isolation & Admin Approval	**Test Execution Date:** 24/10/2026
Description: Exercise facility onboarding workflow: turf owners submit new facility details, the listing enters a pending review queue, and only goes live upon platform administrator approval.	
Pre-Condition: Authenticated Turf Owner account and active Super Admin account; clean turf moderation queue.	

#	Test Step	Test Data	Expected Result	Actual Result	Status
1	Owner submits venue registration form	name: 'Green Field Arena' city: 'Kochi'	HTTP 201 Created; Record created with is_approved=False / status=PENDING	HTTP 201; Confirmation alert: 'submitted for approval' rendered	**PASS**
2	Verify unapproved venue hidden from public catalog	GET /api/turfs (Public view)	Unapproved venue is excluded from public search and listings	Public API query returns only verified/approved facilities	**PASS**
3	Super Admin inspects pending onboarding queue	Admin JWT Token GET /api/admin/pending-turfs	HTTP 200 OK; Returns list of unapproved facilities awaiting review	HTTP 200; Green Field Arena present in moderation table	**PASS**
4	Admin executes facility approval	POST /api/admin/turfs/{id}/approve	HTTP 200 OK; is_approved toggles to True; Listing goes live	HTTP 200; Approved flag updated; Facility immediately searchable	**PASS**

Post-Condition: Facility is published on the live discovery feed; owner granted permission to generate calendar pitch slots.